SEDIC 2026: Maritime Vessel Detection System

Technical Specification Report & End-to-End AI Process Flow

Project: SEDIC 2026 Vessel Detection	**Date:** September 13, 2026
AI Architecture: YOLOv8 Nano (yolov8n.pt)	**GPU Hardware:** NVIDIA GTX 1650 Ti (4GB VRAM)
Target Classes: 10 Maritime Vessel Classes	**Training Status:** Stable Optimised Pipeline

1. Executive Summary

This document provides a comprehensive technical breakdown of the system architecture and process flow for the **SEDIC 2026 Maritime Vessel Detection & Recognition System**. The system leverages state-of-the-art Computer Vision and Deep Learning techniques powered by **YOLOv8 (You Only Look Once v8)** to achieve real-time detection, tracking, and multi-class classification of maritime vessels across surveillance video feeds and imagery.

The end-to-end operational pipeline encompasses 6 core interconnected phases: automated zero-shot data labeling, quality assurance and data cleaning, dataset partitioning, hardware-optimized AI model training, quantitative performance evaluation, and real-time video inference deployment.

2. End-to-End System Process Flow

Phase	Phase Name	Primary Script / Tool	Core Processing Function & Output
1	Data Acquisition & Auto-Labeling	autolabel.py / autolabel_images.py	Image acquisition and automated zero-shot labeling via YO
2	Quality Assurance & Data Cleaning	LabelImg & preview.py	Manual inspection and bounding box verification to ensure
3	Dataset Partitioning & Config	prepare_train_val_split()	Automated 80% Train : 20% Validation split and data.yaml
4	AI Model Training & Optimization	train.py (YOLOv8n)	Neural network training, feature extraction, loss calculation
5	Model Evaluation & Weights Export	Ultralytics Engine	Export of best.pt weights, Confusion Matrix analysis, and r
6	Video Inference & Deployment	predict.py	Real-time vessel detection on input video feeds and annot

3. Detailed Phase-by-Phase Technical Breakdown

Phase 1: Data Acquisition & Automated Labeling (Auto-Labeling)

Manually annotating tens of thousands of maritime images is time-prohibitive. Thus, autolabel.py and autolabel_images.py utilize **YOLO-World (Zero-Shot Object Detection)** to automatically annotate raw images.

- **Processing Pipeline:** Raw imagery is fed into the pretrained zero-shot detector conditioned on 10 vessel class prompt descriptions.

- **Generated Output:** Standardized YOLO text annotation files (.txt) containing normalized coordinates:

```
<class_id> <x_center> <y_center> <width> <height>
```

Phase 2: Quality Assurance & Data Cleaning (Data QA/QC)

AI model performance is bound by data quality (*Garbage In, Garbage Out*). A verification phase was implemented to refine auto-generated labels.

- **LabelImg GUI:** Allows human annotators to review, adjust, or delete misaligned or missing bounding boxes.
- **preview.py:** Lightweight CLI visualizer that rapidly overlays color-coded bounding boxes on images without launching heavy GUI applications.

Phase 3: Dataset Partitioning & Configuration (Dataset Split & Config)

The prepare_train_val_split() routine inside train.py automatically organizes flat image and label directories into standard YOLO format:

- **80% Training Set (39,303 images):** Used by the neural network to optimize weights and learn visual features.
- **20% Validation Set (9,825 images):** Held-out dataset used to evaluate model generalization and prevent overfitting.
- **data.yaml:** Master dataset descriptor file mapping directory paths and the 10 target maritime class names.

Phase 4: AI Model Training & Hardware Optimization

This phase forms the core computational engine where the **YOLOv8 Nano (yolov8n.pt)** model is trained. Specific parameter adjustments were engineered to ensure peak stability on an **NVIDIA GeForce GTX 1650 Ti (4GB VRAM)** GPU on Windows:

Parameter	Value	Technical Rationale & Impact
Base Architecture	yolov8n.pt	Ultra-lightweight architecture (3.0M parameters, 8.2 GFLOPs) optimized for high-FPS real
Batch Size	16	Adjusted from 32 to 16 to prevent CUDA Out-of-Memory (OOM) errors on 4GB VRAM.
Image Resolution	512px	Optimal spatial resolution balancing feature resolution against VRAM allocation.
RAM Caching	cache=False	Disabled because caching 49k+ images requires >24GB RAM. Direct disk reading prevent
DataLoader Workers	workers=2	Reduced from 4 to 2 to eliminate IPC pipe crashes during PyTorch multiprocessing spawn
AMP Precision	amp=False	Disabled because GTX 1650 Ti lacks dedicated FP16 Tensor Cores, preventing NaN loss

Mathematical Engine & Training Loop Execution:

1. **Forward Pass:** Input batches pass through Convolutional Neural Network (CNN) layers, extracting multiscale spatial features via C2f and SPPF modules.
2. **Loss Computation:** Computes bounding box regression loss (*CIoU Loss*) and classification loss (*Binary Cross-Entropy Loss*).
3. **Backpropagation & Optimizer:** The MuSGD optimizer updates network weights to minimize total loss across epochs.
4. **Early Stopping (Patience=15):** Halts training automatically if validation mAP fails to improve for 15 consecutive epochs, preventing overfitting.

Phase 5: Performance Evaluation & Model Export

Training artifacts are automatically logged under runs/detect/sedic_vessel_detector_50ep_b16_n/. Key output files include:

- **weights/best.pt:** Serialized model weights achieving the highest mean Average Precision (mAP) score.
- **results.csv & results.png:** Quantitative logs and trend plots for Loss, Precision, Recall, and mAP@0.5 / mAP@0.5:0.95.
- **confusion_matrix.png:** Inter-class confusion matrix identifying classification ambiguities between vessel types.

Phase 6: Real-Time Video Inference & Deployment (predict.py)

The predict.py script loads best.pt to execute end-to-end video inference:

1. Frame Decoding: Input surveillance videos from videos/ are decoded frame-by-frame.
2. Real-Time Detection: Model predicts bounding box coordinates, vessel class predictions, and confidence percentage scores (%).
3. Visual Rendering: Overlays color-coded bounding boxes and text callouts onto the video frames.
4. Video Export: Encodes and exports the final annotated video file (.mp4) for presentation and reporting.

4. Target Maritime Vessel Class Specifications (10 Classes)

ID	Class Name	Category	Visual Identification Features & Characteristics
0	container_ship	Commercial	Large cargo vessels carrying stacked intermodal shipping containers.
1	tanker	Commercial	Oil/chemical tankers featuring flat decks, piping manifolds, or storage tan
2	cargo_ship	Commercial	Bulk carriers, freighters, or general cargo transport vessels.
3	passenger_ferry	Passenger	Water taxis, passenger ferries, cruise liners, or commuter passenger ves
4	yacht	Recreational	Luxury motor yachts, private sailing yachts, or pleasure craft.
5	speedboat	High-Speed	Small high-speed motorboats, Rigid Inflatable Boats (RHIBs), or jet boats
6	fishing_boat	Fishery	Commercial trawlers, traditional fishing boats, sampans, or longline vesse
7	military_vessel_local	Naval	Domestic naval or Coast Guard vessels (grey hull, pennant numbers).
8	military_vessel_foreign	Naval	Foreign warships, destroyers, frigates, corvettes, or aircraft carriers.
9	military_vessel_general	Naval	Unidentified, general, or auxiliary naval military vessels.

5. Technical Conclusion & Summary

The SEDIC 2026 Maritime Vessel Detection System delivers a robust, scalable, and fully automated computer vision pipeline. By adopting YOLOv8 Nano, the system achieves an ideal balance between real-time inference speed and high detection accuracy.

The hardware optimizations engineered in train.py (batch size 16, disabled RAM caching, controlled worker threads) allow full pipeline execution on mid-range workstation hardware (NVIDIA GTX 1650 Ti 4GB VRAM) without memory

fragmentation or crash faults.